AegisBallot: Aadhaar-Verified Electronic Voting on an Auditable Hash-Chained Ledger

Mohammed Anwar-Ul-Haq
Roll No. 23K91A05F6
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[email]

Mohammad Mosin
Roll No. 23K91A05F4
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[email]

Manda Akshitha
Roll No. 23K91A05E4
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[email]

Kothlapur Ganesh
Roll No. 24K95A0516
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[email]

B Shruthi
[DESIGNATION], project guide
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[email]

***Abstract*---**Electronic voting must ensure that each eligible person votes once, that no recorded ballot is altered, and that the result can be checked. This paper presents AegisBallot, a local web application that binds a ballot to a voter through an Aadhaar-style check (12 digits, Verhoeff checksum, salted-hash registry lookup on synthetic data), issues a single-use token, encrypts the choice with AES-256-GCM and commits it as an Ed25519-signed block approved by two of three validators on a hash-chained ledger. Administrator, voter and auditor consoles are served by FastAPI and React. All 17 automated tests and 25 of 25 HTTP security checks passed. A benchmark of 1000 synthetic voters completed every vote, at 44.9 ms per vote on a loaded PC and about 16-17 ms on an idle one, and a whole voter journey over HTTP took 161.2 ms. A tamper study found that the ledger's own integrity check detects 7 of 13 kinds of tampering and misses edits to the tally table and re-hashed edits, which a stricter audit catches. The ledger is a single-machine simulation, ballot secrecy is partial and TLS is absent.

***Index Terms*---**Aadhaar verification, audit logging, blockchain, election integrity, electronic voting, hash chain, role-based access control.

I. INTRODUCTION

Elections depend on public confidence that every vote is counted exactly once and cannot be altered [2]. Manual polling costs queues, staff and counting time, while a conventional electronic system keeps ballots in a database that an administrator could edit without trace. Blockchain offers an append-only record in which each block commits to its predecessor by a hash [3], and identity binding, such as the Aadhaar number in India [4], offers a way to enforce one voter, one vote. Deployed internet voting has nevertheless been broken in practice, as analyses of Estonia's system [5] and of the Voatz mobile application [6] show, so a design must be judged by what an attacker with database access can still do, not only by its description.

The base paper [1] prototypes a four-tier Ethereum voting model for Oman with biometric voter authentication and measures registration and voting transactions on a Geth test chain. Proposals that combine Aadhaar-style identity with a ledger [7] report a concept and a few blocks but no performance or tamper evaluation. The gap this work addresses is a deployable system that joins identity verification, a ledger, an administration workflow and an auditor, and that is measured, including where it fails. The contributions are:

(1) AegisBallot, a working admin/voter/auditor platform with Verhoeff-checked Aadhaar verification, single-use tokens, AES-256-GCM ballots and a three-validator, quorum-of-two ledger. (2) Four independent layers that block a second vote. (3) Measurements of vote latency and throughput for 1000 synthetic voters, of ledger-check cost against chain length, and of end-to-end HTTP latency. (4) A thirteen-case tamper study that shows what the ledger check detects and, equally, what it misses. Section II surveys the literature, Section III gives the method, Section IV the results and Section V concludes.

A. System Hardware

All measurements ran on one desktop PC: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, no GPU, Windows 11 Pro. The machine was shared with other jobs, so about one third of the CPU was busy when the benchmark started; absolute latencies are therefore upper-ish bounds and vary between runs.

B. Software Requirements

Python 3.11.9; FastAPI 0.110.0 with SQLite through SQLAlchemy 2.0.30; cryptography 42.0.7 (AES-GCM, Ed25519, SHA-256); argon2-cffi 23.1.0 (Argon2id [8]); React 18.3, React Router 7, TanStack Query 5, Tailwind CSS 4 and Recharts 3 built with Vite 8 on Node 24; pytest 9.1. No component contacts an external service.

II. LITERATURE SURVEY

Base paper. Shaikh et al. [1] describe four tiers: MySQL and Ethereum nodes for storage, a smart-contract tier, a C# web tier for administrators and client applications on devices that hold crypto wallets. The contract registers voters and candidates, accepts one vote per registered voter and returns counts. Using synthetic national-ID records for 1000 voters and a Geth test chain on an i7-12700 with 16 GB, the authors time registration and voting transactions run synchronously and asynchronously in batches of 10, 20, 30 and 50. Read from their Fig. 6 and Fig. 9, a registration takes about 2 s and a voting transaction about 3-4 s (the paper gives charts, not

tables). They conclude that blockchain improves integrity and call for pilots, a public-trust study and Layer-2 scaling, noting Ethereum's theoretical 119.1 tx/s. Its limits are synthetic data, no real-world pilot, a public-chain setting with per-transaction delay, and no administration or audit tooling beyond monitoring.

Identity-bound voting. Chouhan and Sharma [7] hash an Aadhaar number or driving licence with BLAKE2b-512, add a fingerprint hash and a picture-rotation pattern, combine them with SHA-256 and record the vote hash in a chain; they show example blocks and an avalanche test whose result they find below the desired 50 %. Krishan et al. [9] propose a permissioned ledger with smart contracts and hashing for anonymity, and acknowledge scalability as open. Bramasto et al. [10] build a decentralized application on the Vexanium blockchain that passed black-box tests but implements only smart contracts.

Reviews and analyses. Jafar et al. [11] find privacy protection and transaction speed to be the most frequently raised issues; Ohize et al. [12] survey architectures and name cybersecurity risk, resource intensity and infrastructure needs as obstacles; Aswini [13] lists malware, server compromise, insiders and limited verifiability and points to hybrid designs with zero-knowledge proofs, homomorphic encryption and sharding; Ray et al. [2] study blockchain-powered voting. Real systems were examined by Springall et al. [5] (Estonia), Specter et al. [6] (Voatz) and, with open audit, Adida's Helios [14]. Lloyd et al. [15] show that on-chain voting can still hide who decides, through metagovernance among DAOs.

TABLE I
REPRESENTATIVE RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	Ethereum, Geth, C#, biometrics	1000 synthetic voters	~2 s reg., ~3-4 s vote (read from charts)	No pilot; public-chain delay
[7]	Aadhaar hash + biometrics, hash chain	Example blocks	Avalanche test < 50 %	No throughput or tamper test
[9]	Permissioned ledger, contracts	None reported	Qualitative claims	Scalability open
[10]	DApp on Vexanium	Black-box test	Tests passed	Contracts only
[5]	Analysis of Estonian i-voting	Live system	Attack paths found	Not a blockchain design
[6]	Analysis of Voatz app	Live app	Weaknesses found	Mobile app, closed
[11]	Review of blockchain voting	Published proposals	Privacy, speed most cited	No implementation
[13]	Review of trust and security	Literature	Malware, insiders, verifiability	No implementation
[14]	Open-audit web voting (Helios)	Deployed system	Ballot audit by voters	No ledger, no national ID

Lessons for the design. Three points follow from the survey. First, identity binding needs a registry that can be checked without exposing the identifier, which argues for hashing with a secret and a checksum before any lookup. Second, the cost the base paper reports per transaction comes largely from a public-chain setting; a permissioned ledger with a small validator set removes mining and gas but also removes decentralisation, so the claim must be limited to local, auditable integrity. Third, deployed systems [5], [6] failed in the surrounding software and keys rather than in the hash chain, so the evaluation here attacks the database, the key files and the checks themselves, not only the chain format.

Consensus and cost. Public chains such as Ethereum [3] settle transactions by network-wide consensus, which gives public verifiability at a cost the base paper quantifies in seconds per transaction and in a theoretical 119.1 tx/s; permissioned platforms and Byzantine-fault-tolerant protocols [16] trade openness for speed by limiting who validates. AegisBallot goes one step further and puts all validators on one machine, which is why its results are reported as an integrity and workflow study rather than as a decentralisation result.

Gap. The surveyed blockchain designs establish integrity models, but none combines national-ID-style identity, a complete election workflow and an auditor in one deployable system, and none reports what its integrity check would miss when an attacker can write to the database. That is the gap AegisBallot measures.

III. METHODOLOGY

Architecture. Fig. 1 shows the tiers. A React single-page application calls FastAPI routes under /api with a session cookie and an X-CSRF-Token header; services implement identity, voting, cryptography and the ledger; one SQLite database holds thirteen tables, and three key files (election key, validator keys, system key) live in data/. Roles are administrator, voter and auditor.

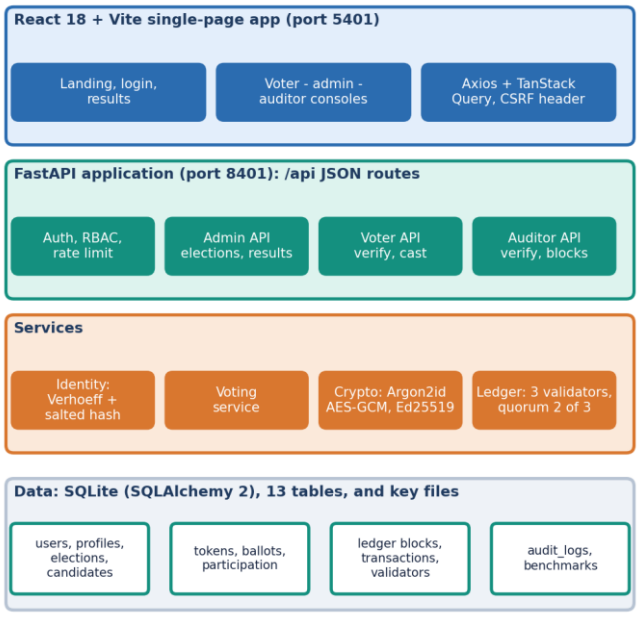


Fig. 1. AegisBallot architecture: React tier, FastAPI tier, service tier with the local ledger, and SQLite storage.

Data. There is no external dataset. The seed script creates one administrator, one auditor, twenty voters with generated Aadhaar-format numbers whose last digit is a valid Verhoeff check digit, two elections, a genesis block and three validators with Ed25519 key pairs; the benchmark generates further synthetic voters with Argon2id-hashed passwords. No real personal data is used anywhere.

Identity. The submitted number must have 12 digits and satisfy the Verhoeff dihedral-group check [17] (a checksum of the digits, c = d(c, p(i mod 8, n_i)) folded over the digits from the right, valid when c = 0). It is then hashed as SHA-256(salt || pepper || number) and looked up in the registry of synthetic voters; only the hash and a masked form are stored. The match must belong to the signed-in account and the voter must be eligible and not yet recorded as voted. The registry is a local synthetic table; UIDAI is not contacted and no OTP or biometric is used.

Token and ballot. A successful check issues a 256-bit random token whose SHA-256 is stored with a ten-minute expiry; it can be used once. The candidate id is encrypted as nonce || AES-256-GCM(K, id) [18] with a 12-byte random nonce and one server-held key. The transaction hash is

$$h_{tx} = \mathit{SHA\text{-}256(canonical\ JSON\{election,\ ballot\ id,\ ciphertext,\ nonce,\ time\}),} \quad (1)$$

signed with Ed25519 [19]. A second vote is blocked at four places: the eligibility row, a unique (election, voter) participation row, the single-use token and the uniqueness of the ledger nonce and h_tx.

Ledger. Each vote becomes one block. With the sorted transaction hashes T, the code computes

$$M = \mathit{SHA\text{-}256(concat(sort(T)))},\quad h_b = \mathit{SHA\text{-}256(canonical\ JSON\{index,\ time,\ } h_{prev}, M, \mathit{sort(T)\}).} \quad (2)$$

M is a flat hash, not a binary Merkle tree [20], so no inclusion proofs exist. Each of three validators checks the transaction schema, a new nonce, a new h_tx and the signature, and signs h_b with Ed25519; the block is committed when at least two approve, a quorum rule in the spirit of Byzantine-fault-tolerant protocols [16] but without any network messaging: all three validator keys are held in-process on one server. Fig. 2 gives the flow. The auditor's validate_chain recomputes M, h_b and the previous-hash link of every block.



Fig. 2. Main flow of one vote, from sign-in to receipt and audit.

Casting procedure. In pseudocode the vote path is: (a) reject unless the election is OPEN and the candidate is active in it; (b) hash the presented token and reject if it is unknown, used or expired; (c) reject if a participation row exists; (d) encrypt, hash and sign; (e) call propose_block, which raises an error unless two validators approve; (f) in one commit store the ballot, mark the token used and add the participation row. Because steps (a)-(c) precede any write and step (f) is a single database transaction, a rejected request leaves no partial state, and a failed block leaves the token usable.

Implementation. The React application has twelve page components (landing, login, results, not-found, four administrator pages, two voter pages and two auditor pages) and talks to 21 JSON routes grouped by role; the original Jinja routes and three helper endpoints remain. The administrator creates an election and every registered voter becomes eligible automatically; closing an election publishes its results to all signed-in roles. About 2,100 non-blank lines of Python implement the backend and 5,300 lines of TypeScript the interface.

Privacy design. The ballots table carries a random ballot id, the ciphertext and the nonce but no voter column, and the tally decrypts ballots only to count them. The intended separation is identity (registry), authorisation (token), anonymous ballot and ledger transaction. In the code, however, the participation table that enforces one vote stores the voter id next to the ballot id, so anyone holding the database and the single AES key can link a voter to a choice; secrecy rests on access control, not on cryptography.

Threat model. The design defends against a voter who tries to vote twice or as someone else, a client that replays or forges tokens and requests, and a casual reader of the ledger. It does not defend against an administrator or intruder with write access to the database and the key files, against denial

of service, or against network eavesdropping, since traffic is plain HTTP on localhost.

Security services. Passwords use Argon2id; sessions are server-side with an HttpOnly, SameSite=lax cookie and no Secure flag; unsafe methods need the CSRF header; logins are limited to 10 per minute and verifications to 15 per minute per client IP; and every login, verification, cast, status change and audit action writes an audit-log row. The Jinja pages of the original build remain but the React app uses the JSON API.

Evaluation protocol. (i) pytest on a scratch database; (ii) an HTTP matrix of 25 security and functional checks against a live server; (iii) the product's own benchmark with 1000 synthetic voters, synchronous and in batches of 10, 20 and 30 (the code processes batches sequentially, so they are not concurrent); (iv) 300 further votes on a growing chain with timing of validate_chain and of each stage; (v) 27 voters through HTTP; and (vi) thirteen tampering cases applied inside a rolled-back transaction, each judged by validate_chain and by a stricter audit written for the study that also verifies validator and transaction signatures, recomputes transaction hashes and cross-checks the ballots table with the ledger.

IV. RESULTS

Test conditions. One shared PC (Section I-A), in-process timings unless stated, synthetic identities only. The system contains no learned model, so accuracy and F1 do not apply; the evidence is functional correctness, tamper detection and latency. Results are single runs; repeated runs differed by the machine load, which is shown below rather than hidden.

TABLE II
MEASURED RESULTS

Measurement	Setting	Result
Unit tests	pytest, scratch DB	17 of 17 passed (5.41 s)
HTTP checks	RBAC, CSRF, replay, Verhoeff, 429	25 of 25 as expected
Benchmark, sync	1000 voters, loaded PC	22.3 votes/s; mean 44.9, median 29.4, p95 75.0 ms; 1000/1000 ok
Benchmark, batches	10 / 20 / 30 (sequential)	22.5 / 37.5 / 22.3 votes/s
Benchmark, idle PC	50-500 voters, Sept. runs	57.7-62.2 votes/s; 16.1-17.3 ms
Stage cost per vote	mean of 60 votes (ms)	token 11.0; crypto 0.53; block 16.8; persist 9.5
validate_chain	26 / 101 / 301 blocks	11.1 / 52.3 / 146.2 ms
HTTP, per voter	27 voters, 127.0.0.1	login 97.9; verify 27.8; cast 35.1; total 161.2 ms
Tamper detection	13 cases, 361 blocks	validate_chain 7; stricter audit 13

Functional and security results. All 17 tests passed, covering the Verhoeff check, unknown and ineligible voters, duplicate votes, role routes, tampered blocks, duplicate transactions, expired tokens, identity separation, audit logging and the encryption round trip. In the live matrix, unauthenticated calls returned 401, role violations 403, a missing CSRF header 403, a bad checksum, an unregistered number, a forged token and a replayed token 400, another voter's valid Aadhaar number 403, and the eleventh login attempt within a minute 429; an injection-style user name was rejected with 401.

Performance. On an idle machine one vote cost 16-17 ms (about 60 votes/s). In the 1000-voter run on the loaded machine the mean was 44.9 ms, with a 654.8 ms maximum, and the three "batch" runs ranged from 22.3 to 37.5 votes/s; since the batches are sequential this spread is machine noise, not an effect of batching. Of a vote's 37.9 ms, the cryptography is 0.53 ms; the rest is database work and the three validators' checks (0.18 ms per Ed25519 verification, 83 ms per Argon2id verification, which dominates the 97.9 ms login). Per-vote cost did not grow measurably over 300 votes (first 100 votes 33.4 ms, last 100 35.0 ms), but validate_chain grows linearly, about 0.5 ms per block (146.2 ms at 301 blocks), because each block issues its own queries.

Run-to-run variation. The same code gave 57.7-62.2 votes/s on five runs in September on an idle machine (50 to 500 voters, mean 16.1-17.3 ms per vote), against 22.3 votes/s in the 1000-voter run while other jobs used the CPU. The maximum single-vote latency of 654.8 ms in that run is most likely a scheduling stall on the shared PC rather than a property of the ledger. These numbers therefore bound the cost of a vote rather than predict it for another machine; in all three measured runs the block commit was the largest stage of a vote and the cryptography the smallest.

Tamper study. Fig. 3(b) lists the thirteen cases. validate_chain caught replaced block hashes, previous hashes, Merkle roots, timestamps and transaction hashes, deleted transactions and a deleted middle block. It missed six: a swapped ledger payload, an edited ballots table (which changes the tally, since results are read from that table), removal of the newest block, forged validator signatures, and both re-hashed edits (T12, T13). The stricter audit detected all 13, at 0.79 s on 361 blocks against 146.2 ms for the product check on 301. Because the validator keys sit on the same disk, an attacker who also reads data/validators.json could forge the signatures the stricter audit relies on.



Fig. 3. Results chart. (a) Mean cost of the stages of one vote and of the HTTP steps. (b) Tamper study: which of thirteen cases are detected by the product's validate_chain() and by a stricter signature and cross-table audit.

What the study means. The ledger format itself behaves as designed: any change to a block that leaves its neighbours alone is found, and the first bad block is named. The weakness is in what the check covers. It reads transaction hashes as stored instead of recomputing them from the transaction fields, never looks at the validator signatures that make the quorum meaningful, and does not compare the table the tally reads with the ledger. A stricter audit that adds these three steps costs roughly four times as much per block (about 2.2 ms against 0.5 ms), which is acceptable for an offline audit but not for every page view; the product runs the cheap check on the public overview, the administrator dashboard and the auditor page. Even the stricter audit assumes the validator keys were not copied, which the single-machine design cannot promise.

Reproducibility. Every number above comes from scripts in the repository (run_experiments.py, the product's run_benchmark, pytest), run against scratch databases so the demonstration data was untouched; the JSON outputs are stored with the code. The HTTP figures exclude the browser and the Vite proxy, and the in-process figures exclude the network entirely.

Output screens. Fig. 4 shows the receipt after a vote: ballot id, transaction hash, block number and block hash, taken from the running application with 200 synthetic voters and 213 blocks. Fig. 5 shows the auditor's console: the chain verified from genesis, the three validators with the two-of-three rule, and the tamper demonstration, which only compares a hash in a cloned copy and does not re-run the integrity check.

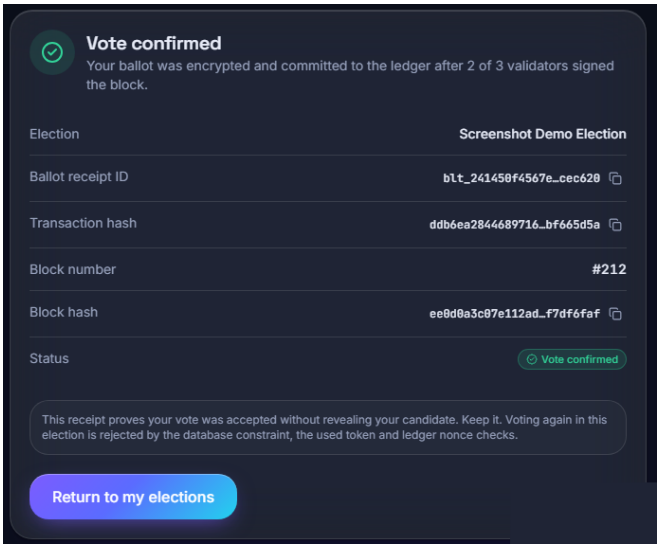


Fig. 4. Output screen: voter receipt after a recorded vote (block #212).

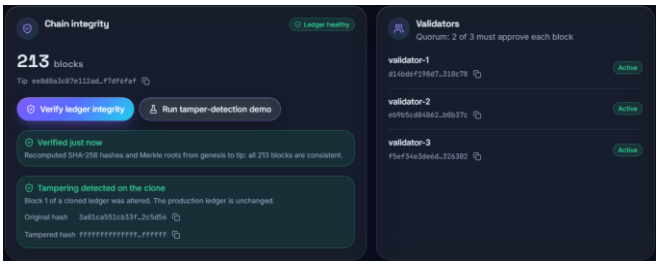


Fig. 5. Output screen: auditor console with chain integrity, validators and the tamper-detection demonstration.

Objectives against outcomes. Identity verification with a checksum and registry: met on a synthetic registry, with no registration function. One voter one vote: met, tested by duplicate-vote, replay and wrong-identity cases. Immutable recording: met only in the weak sense that edits to blocks are detected when they leave the neighbours alone; the six missed cases show it is not tamper-proof against a database writer. Administration portal: met for elections and candidates, while polling windows are stored but not enforced. Results with a chronological audit trail: met. RBAC and input validation: met; TLS: not met; integrity and performance evaluation: done, with the limits stated in this section.

Comparison with the base paper. The base paper's per-transaction times (about 2 s registration, 3-4 s voting, read from its charts) were measured on a Geth test chain with C# clients, biometric authentication and a different i7-12700 machine; AegisBallot's mean in-process vote took 44.9 ms on a loaded PC and 35.1 ms for the cast call over HTTP. These are orders of magnitude apart, but they are not comparable: a local simulated ledger without network consensus, mining or gas does far less work, and gives none of the decentralisation or public verifiability that a public chain provides. The datasets also differ (both synthetic, 1000 voters in the benchmark) and the definitions differ (the base paper times registration and vote separately, here a vote includes token issue).

Failure cases and limits. (1) The integrity check misses six of thirteen tamper cases. (2) Ballot secrecy is partial: the participation table stores voter and ballot ids together, one AES key protects all ciphertext, and an administrator can read a live tally of an open election. (3) The cookie has no Secure flag and no TLS is configured, although the batch objectives call for TLS. (4) The login limiter is per IP, so users behind one address share 10 logins a minute. (5) Concurrent voting, a penetration test and a user study were not carried out. (6) The registry is synthetic; real Aadhaar integration is out of scope and its legal limits [4] apply.

V. CONCLUSION AND FUTURE SCOPE

AegisBallot meets the batch objectives of Aadhaar-style verification, one voter one vote, an immutable-by-design ledger, an administration portal, results with an audit trail and RBAC, and it measures its own integrity and performance; the objective of TLS is not met. Its key finding is that a hash-chained ledger is only as trustworthy as the check run over it: the shipped check detects 7 of 13 tamper cases while a

stricter audit detects all. Limits are the single-machine ledger, partial secrecy, synthetic identities and a shared test PC. Future work is to verify signatures, transaction hashes and the ballots table in the product, use a true Merkle tree and per-election keys, separate voter and ballot records, add TLS, a second factor and network validators with a real consensus protocol [16], and run concurrent-load, penetration and usability tests.

REFERENCES

[1] A. Shaikh, N. Adhikari, A. Nazir, A. S. Shah, S. Baig and H. Al Shihi, "Blockchain-enhanced electoral integrity: a robust model for secure digital voting systems in Oman," *F1000Research*, vol. 14, art. 223, version 3, Sep. 2025, doi: 10.12688/f1000research.160087.3.

[2] M. Ray, V. Ojha and W. Rhmann, "Electronic voting system powered by blockchain technology: a study," *SSRN Electronic Journal*, 2024, doi: 10.2139/ssrn.4483852. [SSRN lists the posting as May 2023 (ICCS 2023); the batch records 2024.]

[3] V. Buterin, "A next-generation smart contract and decentralized application platform (Ethereum white paper)," 2014.

[4] Government of India, "The Aadhaar (Targeted Delivery of Financial and Other Subsidies, Benefits and Services) Act, 2016," The Gazette of India, Act No. 18 of 2016.

[5] D. Springall *et al.*, "Security analysis of the Estonian internet voting system," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2014, pp. 703-715.

[6] M. A. Specter, J. Koppel and D. Weitzner, "The ballot is busted before the blockchain: a security analysis of Voatz, the first internet voting application used in U.S. federal elections," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1535-1553.

[7] S. Chouhan and G. Sharma, "A new era of elections: leveraging blockchain for fair and transparent voting," arXiv:2502.16127, Feb. 2025.

[8] A. Biryukov, D. Dinu and D. Khovratovich, "Argon2: new generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, 2016, pp. 292-302.

[9] K. Krishan, S. Raj and P. Raj, "Decentralized trust: a blockchain-based framework for secure electronic voting," SSRN preprint, Mar. 2026.

[10] S. Bramasto, S. F. Savitri and E. R. Djuwitaningrum, "Electronic voting (e-voting) sebagai aplikasi terdesentralisasi pada Vexanium blockchain," *Faktor Exacta*, vol. 17, no. 2, pp. 96-107, Jun. 2024, doi: 10.30998/faktorexacta.v17i2.19405.

[11] U. Jafar, M. J. Ab Aziz and Z. Shukur, "Blockchain for electronic voting system - review and open research challenges," *Sensors*, vol. 21, no. 17, art. 5874, 2021.

[12] H. O. Ohize *et al.*, "Blockchain for securing electronic voting systems: a survey of architectures, trends, solutions, and challenges," *Cluster Comput.*, vol. 28, no. 2, art. 132, 2025 (online Nov. 2024), doi: 10.1007/s10586-024-04709-8.

[13] T. Aswini, "Blockchain in electronic voting systems: trust and security challenges," *Sci. J. Artif. Intell. Blockchain Technol.*, vol. 2, no. 4, pp. 69-77, Oct.-Dec. 2025, doi: 10.63345/sjaibt.v2.i4.108.

[14] B. Adida, "Helios: web-based open-audit voting," in *Proc. 17th USENIX Security Symp.*, 2008, pp. 335-348.

[15] T. Lloyd, D. O Broin and M. Harrigan, "The on-chain and off-chain mechanisms of DAO-to-DAO voting," arXiv:2603.00708, Feb. 2026.

[16] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. 3rd Symp. Operating Syst. Design Implement. (OSDI)*, 1999, pp. 173-186.

[17] J. Verhoeff, "Error detecting decimal codes," Mathematisch Centrum, Amsterdam, The Netherlands, Math. Centre Tract 29, 1969.

[18] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[19] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," IETF RFC 8032, Jan. 2017.

[20] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology - CRYPTO '87*, ser. LNCS, vol. 293. Berlin, Germany: Springer, 1988, pp. 369-378.